

Sofra Frontend Engineering Case Study

Case Study · 2026

Overview

Sofra is a fictional food delivery application. Its assistant lets users work with Sofra through natural language: the user types "order 2 cheeseburgers", and the assistant answers with a **model-generated interface**, a typed JSON document describing cards, summaries, confirmation prompts and gates. It does not answer with free text.

The assistant backend already exists; we give you a mock of it. In this case study you will build **the client**: the web app that streams those documents, renders them, and puts a human in charge of every action that spends money.

We are not looking for pixel-perfect screens alone. We want you to solve the hard parts of a real client for a model-driven interface, at small scale: rendering output you cannot trust, streaming over an unreliable transport, and making sure money moves only when a person actually meant it.

Time: one weekend, roughly 3 days

Deliverable: Git repository + README (client in client/)

Language: TypeScript (React preferred)

Scenario

The server is driven by a language model. It is usually right, and it is sometimes wrong in ways you can predict. It may emit a component the client has never heard of, send a price as a string, drop the connection halfway through an answer, or echo an order note containing HTML back into its markdown. The client is the last layer before a human's eyes and a human's click. You will build that layer:

1. A chat surface that consumes a **streamed** response and renders it progressively, without corrupting it and without losing track of which turn it belongs to.
2. A **renderer** for the component catalog that validates everything at runtime and **fails closed**: a block it cannot trust is not drawn as if it were fine.



3. The **confirmation experience** for sensitive actions (placing an order, cancelling one, tipping). A confirmation is executed exactly once, only by a deliberate human action, and only while it is still valid.
4. A display layer that **never invents or recomputes** a number: money, dates and statuses come from the server, and they stay correct after the state changes.

What we give you

Path	Contents
<code>mock-server/server.mjs</code>	The mock assistant backend. Zero dependencies, Node ≥ 20: <code>npm start</code> (or <code>node mock-server/server.mjs</code>) and it listens on <code>http://localhost:4000</code> . It is not an LLM. It is a deterministic, scripted stand-in that understands the prompts in this README, speaks the real contract, enforces the confirmation rules for real, and keeps state (wallet, cart, orders) that changes when actions execute.
<code>schema/ui_spec.schema.json</code>	The response contract: the only format the assistant returns. The backend case study uses the same contract, with three optional fields added here (<code>audit.kb_doc_ids</code> , <code>order_summary.date</code> , <code>order_summary.note</code>).
<code>schema/stream_events.schema.json</code>	The streaming protocol: one JSON event per line.
<code>data/*.json</code>	The data behind the mock: 4 users, 11 restaurants with menus, carts, orders. You do not need to read these files; everything is available over the API. They are here so you can tell what the right answer looks like.
<code>data/knowledge/kb.jsonl</code>	The help-center knowledge base, about 2,100 documents, served by <code>/api/kb/*</code> . Only the bonus help-center and sources features need it.
<code>scenarios.jsonl</code>	The scenario table below in machine-readable form, including the ledger assertions we check.
<code>scripts/check-ledger.mjs</code>	<code>npm run check -- <id></code> evaluates a row's ledger assertions against the running mock; <code>npm run reset</code> resets it. See "How to read <code>scenarios.jsonl</code> ".

All of the data is synthetic.

The mock reads a few optional environment variables: `PORT` (default 4000), `TOKEN_TTL_SECONDS` (300), `STREAM_DELAY_MS` (18, delay between network chunks), `REST_LATENCY_MS` (250), `CHAOS` (see "Chaos modes") and `PUBLIC_ORIGIN` (the URL your browser reaches the mock at, default `http://localhost:<PORT>`; the security payloads in the data point at it, so set it if you serve the mock from another host or through a proxy). We review with the defaults.

No design is provided. Visual design is part of the evaluation. We are not grading taste. We are grading whether the interface communicates. The clearest test: a `verification_gate` (blocked), a `confirmation_prompt` (waiting for you), an `order_summary` of a completed order (done) and an `error` must be **impossible to confuse at a glance**, without relying on colour. Two more things we look at: the layout does not jump while text streams in, and the page holds together in a desktop browser. We review on a desktop; a phone layout is welcome, not required.

The server's clock is not your clock. The mock's "now" starts at **2026-08-20T12:00:00+03:00** and advances in real time. Every response carries it: `server_now` in the stream's `meta` event, and an `X-Sofra-Now` header on every API response. Confirmation expiry, "2 days ago" and "delivered today" are all relative to the server's clock. A client that compares `expires_at` with `Date.now()` will think every confirmation expired a month ago. Calendar dates such as `order_summary.date`, and the server's "today", are Europe/Istanbul dates; `server_now` itself is UTC.

The task

Must-have

1. **Streaming chat.** Send messages to `POST /api/chat`, consume the NDJSON stream and render blocks as they arrive. Read "The stream" below: network chunks do not align with lines or with UTF-8 characters, events can arrive twice, and a stream can end without `done`. Users can stop a response, and can send a new message while one is streaming. Nothing from an old stream may ever render into a newer turn. Send the `conversation_id` from the `meta` event back on every later turn: follow-ups such as "make it 5 cheeseburgers" and "include my cart" only work inside a conversation, and in a fresh one the mock answers with a clarification instead.
2. **The renderer.** Render every component in the catalog. Validate every block at runtime before rendering it. An **unknown** block type is skipped gracefully, and the rest of the response still renders. An **invalid** block is never drawn with wrong data. A `confirmation_prompt` that fails validation must **never** produce an actionable Confirm control (fail closed). How you validate (the JSON schema itself, a hand-written parser, Zod generated from the schema...) is your decision; explain it.
3. **The confirmation experience.** Implement the lifecycle of a `confirmation_prompt` as described under "Sensitive actions": `live` → `confirming` → `done`, and the ways it can end otherwise (expired, superseded, rejected, outcome unknown). A confirmation executes **exactly once** per deliberate human action: no double submits, no confirming by pressing Enter in the composer, no confirming a prompt that has been replaced.
4. **The server is the source of truth, and the shell shows it.** The app is more than the chat: a header names the current user and shows their wallet balance, and the cart and the order list are visible somewhere, all fed by the plain REST reads below. Display money, dates and statuses as the server sends them. The client does not add up prices, apply delivery fees or decide whether a minimum is met. After an action executes, anything on screen that it changed (the wallet balance in the header, the order list, the cart) must be correct without a manual reload.
5. **Safe content.** `text.markdown` is written by a model that sometimes quotes untrusted data verbatim. Order notes are written by end users. Neither may execute script, load remote resources on its own, or produce a link that runs code. See "Rules".
6. **An audit inspector.** Every response carries an `audit` record (`decision` and `reason` always; `intent`, `tools_called` and `kb_doc_ids` when present). Show it in a developer-facing panel or drawer. We use it during review, and you will find it useful while building. Its `decision` value (`clarify`, `unknown`, `blocked`...) is also information the *user* may deserve. How much of it you surface in the product UI is a design decision; explain it. The renderer's validation failures (rule 2) belong in the same place.

7. **Tests for the riskiest parts.** At minimum, the stream parser and the confirmation lifecycle. We would rather see ten tests that pin down the dangerous behaviour than a hundred snapshot tests. These are the behaviours a test should pin down:
- a UTF-8 character split across two chunks decodes intact;
 - a line split across two chunks is parsed once;
 - a duplicated `seq` is applied once;
 - a stream that ends without `done` is marked incomplete;
 - an `error` event ends the turn and exposes `retryable`;
 - a stream stopped by the user, or superseded by a new message, renders nothing into a later turn;
 - a double click or a held Enter on Confirm sends exactly one request;
 - a prompt past `expires_at`, by the server's clock, cannot be confirmed;
 - a newer prompt for the same action makes the older one inert;
 - an execute request that dies without a response reconciles through `GET /api/actions/status` and never asks for a second approval;
 - a `confirmation_prompt` that fails validation never renders a Confirm control.

Bonus (we do not expect all of these)

- **End-to-end tests** (Playwright or similar) that run the scenario table against the mock and assert on `/__admin/ledger`.
- **Resuming after reload.** Restore a conversation from `GET /api/conversations/:id`. Pending confirmations must come back in their *real* state: ask `GET /api/actions/status`; do not trust what is in `localStorage`.
- **A help-center screen** over `GET /api/kb/search` (2,100 documents, paginated). The knowledge base contains archived policies, stale announcements and support tickets that contradict current policy. Showing a reader which document to trust is the interesting part, and the data does not make it easy on purpose: there is no `archived` field. An archived document may carry an `archive` tag, a "(legacy)" or "(2024 archive)" title, or a `_v0 / _old` id suffix, and 29 FAQ and how-to documents have no date at all. Pay attention to Turkish text: searching `istanbul` should find `İstanbul`.
- **Sources.** Render `audit.kb_doc_ids` as citations that open the document.
- **A component workbench** (Storybook, Ladle, a plain page...) that shows every block in every state, including invalid ones.
- **A screen-reader run** of rows 4–5 (VoiceOver or NVDA) and a note on what is announced and when. Rule 7 says what we hope to hear.



- **A short note** on performance in long conversations (hundreds of blocks, a large text streaming in), bundle size, or what streaming UI costs in layout stability.

The API (mock server)

The mock allows any origin (CORS), so you can call it directly or through a dev-server proxy. There is no authentication: every request names its `user_id`. A user switcher in your UI (or a query parameter) is fine. Conversations belong to a user: continuing another user's `conversation_id` is a 403, and a confirmation issued to one user is `token_invalid` for another. Switching user therefore starts a new conversation, and a live prompt from the previous user must not stay confirmable (row 24).

```

# --- the assistant ---
POST /api/chat { user_id, message, conversation_id? }
  -> 200 application/x-ndjson (see "The stream")
  -> 4xx/5xx application/json { error: { code, message } }
                                429 carries a Retry-After header (seconds)

# --- sensitive actions ---
POST /api/actions/execute { user_id, action, params, confirm_token }
  -> application/json, always a ui_spec response (blocks + audit):
    200 executed the action ran; the response describes the result
    403 token_invalid forged, unknown, or another user's token
    409 token_used this token was already executed
    409 token_superseded a newer confirmation replaced it
    409 token_void a gate closed on an earlier attempt (see 422)
    409 params_mismatch params differ from what the token is bound to;
                        if the params sent are well-formed, the response
                        carries a NEW confirmation_prompt for them
    410 token_expired the first expired attempt carries a NEW
                        confirmation_prompt; later ones do not
    422 gate_closed state changed since the prompt (e.g. balance);
                        the response carries a verification_gate

GET /api/actions/status?confirm_token=...
  -> { state: live|expired|used|superseded|void|invalid,
      action, expires_at, executed_at?, result? }
      result is the original 200 response, present when state is "used".
      "void" means a gate closed at execution time; "invalid" means the
      server does not recognise the token.

# --- plain reads (JSON, with a little artificial latency) ---
GET /api/users [ { id, display_name, district } ]
GET /api/users/:id { wallet_balance_try, payment_method, age_verifie
d, address, district, ... }
GET /api/users/:id/cart { restaurant_id, restaurant_name, items:[...], quot
e }
GET /api/users/:id/orders [ { order_id, date, restaurant, total_try, status,
note, tips_try } ]
GET /api/restaurants?q=&cuisine=&near_district=
GET /api/restaurants/:id/menu
GET /api/kb/search?q=&category=&limit=&offset=
GET /api/kb/:id
GET /api/conversations/:id { turns: [ { role, text | response, at, complete }
] }

                                complete: false means the server itself
                                cut or errored the stream; response is
                                the full document it generated. A
                                stream the user stopped is complete:
                                the server has the whole answer, and a
                                resumed conversation shows it

# --- for you and for us ---
GET /__admin/ledger every execute attempt, every execution, every security beacon hit
POST /__admin/reset restore the initial data and clear all state
POST /__admin/clock { advance_seconds } move the server clock forward

```

A client that follows "Send exactly what was shown" under "Sensitive actions" never triggers `params_mismatch`. It exists so that a buggy or hostile client cannot execute anything other than what was approved; render its response like any other rejection and move on.

The stream

POST `/api/chat` answers with `application/x-ndjson`: one event per line. Applying the events in order produces a document that validates against `ui_spec.schema.json`:

```
{ "seq":1, "event":"meta", "version":"1", "request_id":"rq_...", "conversation_id":"cv_...", "server_now":"2026-08-20T09:00:04.120Z" }
{ "seq":2, "event":"block", "index":0, "block":{"type":"text", "markdown":""} }
{ "seq":3, "event":"text_delta", "index":0, "delta":"Here's your c" }
{ "seq":4, "event":"text_delta", "index":0, "delta":"art from Bur" }
{ "seq":9, "event":"block", "index":1, "block":{"type":"cart_summary", "restaurant_id":"rst_04", "items":[...], "total_try":175} }
{ "seq":12, "event":"audit", "audit":{"decision":"answered", "reason":"data-backed", "tools_called":["get_cart", "quote_checkout"]} }
{ "seq":13, "event":"done" }
```

What the transport does **not** guarantee (the mock does each of these, some on every response):

- **Chunks are not lines, and not characters.** A network chunk can end in the middle of a line, and in the middle of a multi-byte UTF-8 character. Sofra's data is full of them (Kadıköy, Şişli, Üsküdar). A client that decodes each chunk on its own will show Kad~~ıköy~~köy.
- **Deltas split markdown anywhere**, including inside ***bold*** or a link. Your rendering of a half-finished text block should not flicker between interpretations or break the layout.
- **At-least-once delivery.** An event can be delivered again. `seq` is strictly increasing: an event whose `seq` you have already applied is a duplicate.
- **Incomplete streams.** A response is complete only when `done` arrives. The stream can also end with an `error` event (`retryable` tells you whether to offer a retry), or simply stop. What you have received so far stays on screen, marked incomplete. It never looks finished.
- **Version.** The client supports `version: "1"`. Anything else is not rendered at all; tell the user plainly instead.

Chaos modes

Start a message with `/chaos <mode>[, <mode>...]` and the mock misbehaves for that request; the prefix is stripped before the message is interpreted (for example `/chaos slow, replay What is in my cart?`). The same modes can be sent in an `X-Sofra-Chaos` header, or set for every request with the `CHAOS` environment variable. Your client does not need any code for this; the

prefix travels as part of the message. The transport failures (`slow`, `drop_mid_stream`, `error_event`, `http_429`, `http_500`) fire **once** per user and identical message, so retrying the same message succeeds, as a transient failure should.

Mode	What happens
<code>slow</code>	~4 s before the first byte, then a slow stream. Use it to exercise Stop and sending a new message mid-stream.
<code>replay</code>	Windows of already-sent events are delivered again.
<code>drop_mid_stream</code>	The connection is cut about halfway through.
<code>error_event</code>	The stream ends with a retryable <code>error</code> event.
<code>http_429 / http_500</code>	An HTTP error instead of a stream. 429 carries <code>Retry-After: 3</code> ; whether you retry automatically or offer a button is your call, but a retry must not be possible before the window has passed. 500 is a final failure for that attempt: say so plainly and offer a retry, which succeeds.
<code>unknown_block</code>	A block type outside the catalog is inserted.
<code>invalid_block</code>	One block violates the schema (e.g. <code>price_try: "195 TL"</code>).
<code>malformed_confirmation</code>	The <code>confirmation_prompt</code> is missing <code>expires_at</code> . The token behind it is real and would execute: the only thing between it and the user's money is your validator.
<code>version_2</code>	The response declares <code>version: "2"</code> .
<code>short_ttl</code>	Confirmation tokens in this response expire after 20 s instead of 5 min.
<code>drop_execute_response</code>	The next <code>execute</code> call for a token from this response executes , and then the connection is dropped before the response is sent.

Generative UI: the component catalog

`text · restaurant_card · menu_item · cart_summary · order_summary · confirmation_prompt · verification_gate · suggested_actions · error`

The response is always `{ "version": "1", "blocks": [ ... ], "audit": { ... } }`. The fields of each block are in the schema. Notes on the ones with behaviour:

- **text**: CommonMark, streamed. See rule 4.
- **menu_item**: `available: false` and `age_restricted: true` are states the user has to see before they try to order.
- **cart_summary**: shows the server's `subtotal_try`, `delivery_fee_try` and `total_try`. The delivery fee is 0 above 250 TL even though the restaurant card shows the restaurant's own fee. That is policy, and the server has already applied it. `meets_minimum: false` should be visible.
- **order_summary**: `status` is `received`, `delivered` or `cancelled`. `note` is plain text written by a customer.
- **confirmation_prompt**: see "Sensitive actions". The `summary` is human-readable; `params` is what the token is bound to.
- **verification_gate**: the action was stopped and **nothing was executed**. `requirement` says why (`out_of_service_area`, `item_unavailable`, `age_18_plus`, `min_order`, `sufficient_funds`, `not_cancellable`, `tip_window_expired`); `reason` explains it to the user, and `cta` names the way forward.
- **suggested_actions**: chips. Tapping one sends its text as a new user message. That is all a chip can ever do.
- **error**: `code` + `message`.

You may not change the contract the server speaks. If you think the catalog or the protocol is missing something (and there are defensible arguments that it is), write it down in your README with the change you would make. That reasoning is part of the evaluation.

Sensitive actions

`place_order`, `cancel_order` and `add_tip` are never executed by the assistant. The assistant produces a `confirmation_prompt`; the action runs only when the user confirms it **in your UI**, which calls `POST /api/actions/execute` with the prompt's `action`, `params` and `confirm_token`.

What the server guarantees (and the mock enforces):

- The token is bound to an immutable copy of the approved transaction: `user`, `action`, `params` including `amounts`, `expires_at`, and a `nonce`.
- It is **single-use**.
- If the `params` you send differ from the bound ones, it is rejected.
- It is rejected after `expires_at`.
- Only **one** confirmation per user and action is live at a time. Issuing a new one supersedes the older one.

The server's guarantees protect the account. They do not make the *interface* correct. A client that fires two requests on a double click will not create two orders, but it will show the user an alarming "already used" error right after a success. A client that keeps an old prompt's Confirm button active invites the user to approve something they have already changed their mind about. What we expect from the client:

1. **One deliberate action, one request.** Double clicks, a held-down Enter key or a slow network produce exactly one `execute` call per confirmation. We check the ledger.
2. **Confirm only from the prompt.** Nothing but the prompt's own control confirms. Not Enter in the composer, not a suggested-action chip, not a link in markdown. Typing "confirm" in the chat sends a message; the mock will point the user back to the card. Do not auto-focus the Confirm control when a prompt appears: the user is probably still typing, and their next Enter must not spend their money.
3. **Send exactly what was shown.** `params` go back as the object you parsed, unmodified: the client never edits, rebuilds or re-derives them. Key order does not matter (the server compares a canonical form); values do.
4. **Expiry follows the server's clock.** Show the time left, computed against `server_now`, and disable the prompt when it runs out. If a request races the deadline and the server answers 410, the response already carries a fresh prompt: use it.
5. **At most one live prompt per action on screen.** When a newer prompt for the same action arrives, older ones in the scrollbar become visibly inert ("replaced").
6. **Rejections are not failures of the user.** 409/410 responses carry the next step (a new prompt, a gate, an explanation). Render it.
7. **An unknown outcome is not a failure.** If the `execute` request dies without a response (network error, timeout, a dropped connection), the action **may have run**. Do not report failure, do not report success, and do not ask the user to approve again. Find out: GET



`/api/actions/status` returns the real state and, if it ran, the original result. Only one execution may ever exist for one approval.

Rules (what the client must follow)

1. **No invented numbers.** Prices, fees, totals, balances and statuses are displayed as the server sent them. Formatting is yours: `tr-TR` Turkish lira formatting is a reasonable default. Computing is not.
2. **Validate, then render. Fail closed.** A block that fails validation is not rendered as if it were valid: no `NaN` `TL`, no `undefined`, no Confirm button. Unknown blocks do not crash the response. The failure is visible to a developer (console, audit inspector) and handled calmly for the user.
3. **Data is not markup.** `order_summary.note` and anything else that is not `text.markdown` is plain text. Render it as text.
4. **Markdown is untrusted.** In `text.markdown`: raw HTML is not rendered as HTML; only `http(s) :` and `mailto:` links are clickable, and external links are recognisable as such and open safely; images are **not** loaded automatically (render the alt text or a link instead). The mock's data contains payloads that call back to `/__beacon` if any of this goes wrong. The ledger records every hit, so we will know. The case of a model quoting an order note verbatim is realistic, not contrived.
5. **Stale is worse than slow.** After an action executes, the data it changed is refreshed or invalidated wherever it is shown. A header that still says 800 TL after a 390 TL order is a bug.
6. **Every async state is designed.** Loading, streaming, stopped, incomplete, failed-retryable, failed-final, rate-limited. None of them is a blank area or a spinner that never ends.
7. **Accessible by default.** Everything, including confirming an order, works with the keyboard alone, with visible focus, sensible contrast and no information carried by colour alone. For screen readers, covered by the bonus screen-reader run: streaming text is announced sensibly, not token by token, and a gate and a confirmation are announced for what they are.
8. **Turkish is not an edge case.** Names such as `Kadıköy`, `Çay` (samovar) and `Ege Balık` render intact everywhere, including when streamed. If you upper-case or case-fold text, do it with the right locale (`i` → `İ`).

Scenarios you must handle

Reset the mock (POST /__admin/reset, or `npm run reset`) before every row that changes state; those rows carry `fresh_data` in `scenarios.jsonl`. The other rows are independent. Where a row says "after", run it in the same conversation. The mock is scripted, so use the prompts as written: a paraphrase may fall through to "I only understand a fixed set of requests".

#	User	Do	Expected
1	u_ok	"What is in my cart?"	Text streams in, then a <code>cart_summary</code> : subtotal 150, delivery 25, total 175, exactly as sent. No mojobake in <code>Patates Kızartması</code> or anywhere else.
2	u_ok	"Is there a pizza place near me?"	Two <code>restaurant_cards</code> , in the server's order (Napoli Fırın 4.6, then Pizza Locale 3.9).
3	u_ok	"Show the Pizza Locale menu", then "Show the Köfteci Ramiz menu"	<code>menu_items</code> . The energy drink is visibly 18+; Künefe is visibly unavailable.
4	u_ok	"Order 2 cheeseburgers from Burger Stop"	Text stating the server's assumption, a <code>cart_summary</code> (390, free delivery), and a <code>confirmation_prompt</code> with a countdown from the server's clock. The Confirm control is not focused; pressing Enter in the composer does not confirm.
5	u_ok	(after 4) confirm on the card	Exactly one <code>execute</code> . An <code>order_summary</code> for the new order. The prompt becomes inert ("confirmed"). The header wallet goes from 800 to 410 without a reload.
6	u_ok	Repeat 4, then double-click, triple-click, or hold Enter on Confirm	Ledger: exactly one <code>execute</code> attempt for that token. No "already used" error shown after the success.
7	u_ok	After a fresh row 4 (not confirmed): "make it 5 cheeseburgers"	A new prompt (975). The old prompt is visibly replaced and cannot be confirmed. Ledger: no attempt with the old token.
8	u_ok	"/chaos short_ttl Order 1 cheeseburger from Burger Stop", then wait	Countdown reaches zero, Confirm disables, and the user can ask for a fresh confirmation. Also try confirming in the last second: a 410 answer is handled by showing the new prompt it carries. The 20 s wait is real time. POST /__admin/clock is for your own tests; after a jump, a client is expected to pick up the new time from its next response, not before.

#	User	Do	Expected
9	u_ok	"/chaos drop_execute_response Leave a 20 TL tip on my last order", then confirm	The execute request dies, but the tip was added. The UI shows neither a failure nor a new approval request. It reconciles and ends up saying the tip was added. Ledger: exactly one execution.
10	u_lowbalance	"Order levrek from Ege Balık"	verification_gate sufficient_funds. No confirmation control anywhere.
11	u_unverified	"Order 1 energy drink from Ege Balık"	verification_gate age_18_plus.
12	u_new	"Order from Ege Balık"	verification_gate out_of_service_area.
13	u_ok	"Order künefe from Köfteci Ramiz"	verification_gate item_unavailable.
14	u_ok	On fresh data: "Cancel my last order" (u_ok_o9, received), then confirm	A confirmation that reads as <i>destructive</i> . After confirming, the order shows cancelled wherever it is displayed, and the refund is reflected in the wallet.
15	u_ok	"Cancel order u_ok_o1"	verification_gate not_cancellable. No confirmation control.
16	u_ok	"Leave the courier a 50 TL tip on u_ok_o1"	verification_gate tip_window_expired (delivered 2026-07-10, more than 7 days before the server's today).
17	u_ok	"Show my recent orders"	Orders with dates relative to the server's today. One note carries an injected "SYSTEM NOTE"; another carries HTML, javascript: and remote-image payloads, and the assistant quotes both in its markdown. Everything renders as inert text. Ledger: zero security_beacons.
18	u_ok	"How much is the delivery fee currently?" / "What is the calorie value of the Köfte?"	The first is answered and cites pol_delivery_fee_v2 first; the second is unknown. The inspector shows both; whether and how the product UI distinguishes "I don't know" is your call.
19	u_ok	"Show Napoli Fırın on a map"	The response contains a map_view block, which is not in the v1 catalog. The rest of the response renders. No crash, no empty hole.

#	User	Do	Expected
20	u_ok	"/chaos invalid_block Show the Burger Stop menu", and "/chaos malformed_confirmation Order 2 cheeseburgers from Burger Stop"	The invalid <code>menu_item</code> is isolated; the others render. The malformed prompt yields no actionable Confirm. Ledger: zero attempts with that token.
21	u_ok	"/chaos replay ...", "/chaos drop_mid_stream ...", "/chaos error_event ...", "/chaos http_429 ...", "/chaos http_500 ...", "/chaos version_2 ..." (any prompt)	No duplicated blocks or doubled text. Partial answers stay, marked incomplete, with a retry, and the retry succeeds. 429 waits for <code>Retry-After</code> before retrying. 500 is told plainly as a final failure for that attempt, with a retry that succeeds. Version 2 is refused politely.
22	u_ok	"/chaos slow What is in my cart?", then Stop, or send another message mid-stream	The first stream is aborted. Nothing from it lands in the new turn.
23	u_ok	Rows 4–5 using only the keyboard	Reachable and operable, with visible focus, and nothing that depends on colour alone.
24	u_ok, then u_new	"Order 2 cheeseburgers from Burger Stop", then switch the user to u_new without confirming	The u_ok prompt is no longer confirmable. The next message starts a new conversation for u_new. Ledger: zero attempts.

Rows 1–3 and 17–19 show that the right thing renders. Rows 4–9, 14 and 24 are the confirmation lifecycle, each ending in exactly one execution or none. Rows 10–13, 15 and 16 show that a gate is never confused with a prompt. Rows 20–22 show that the client survives the server, and row 23 shows that everyone can use it.

How to read `scenarios.jsonl`

Each line is one row of the table above:

```
{
  "id": "sc_06",
  "user_id": "u_ok",
  "fresh_data": true,
  "area": "confirm",
  "steps": [
    "Order 2 cheeseburgers from Burger Stop",
    "<double/triple click Confirm, or hold Enter on it>"
  ],
  "expect": {
    "ui": [
      "success shown once",
      "no 'already used' error after success"
    ]
  },
  "ledger": {
    "attempts_per_token_max": 1,
    "executions": 1
  }
}
```

- `steps`: what the reviewer types or does, in order. Angle brackets mark an action rather than a message.
- `fresh_data`: reset the mock before this row.
- `area`: render, confirm, gate, security, robustness **OR** ally.
- `expect.blocks`: the exact sequence of block types the mock sends for the last step.
`expect.blocks_after_confirm`: the same for the execute response. `expect.values`: fields checked on the first block of that type. `expect.audit_decision`: accepted values of `audit.decision`. `expect.ui`: what a reviewer checks by eye; these are the things the ledger cannot see.
- `ledger`: assertions over `GET /__admin/ledger`, read after the row, counted since the last reset:
 - `executions`: exact count of executions.
 - `execute_attempts`: exact count of `execute_attempts`.
 - `attempts_per_token_max`: no single token appears in more attempts than this.
 - `outcomes`: for each named outcome, the exact number of attempts with that outcome, for example `{"token_superseded": 0}`.
 - `bait_attempts`: attempts whose token came from a `malformed_confirmation` prompt (`malformed_prompt_bait` in the ledger).
 - `security_beacons`: exact count.

`npm run check -- sc_06` evaluates a row's ledger assertions against the running mock and prints PASS or FAIL per assertion; run it after performing the row in your client. `npm run check` alone prints a summary of the ledger. `npm run reset` resets the mock.

How we review

- We run your client against the unmodified mock, with the default settings, and walk through the scenario table typing the prompts as written, resetting the mock before every row marked `fresh_data`.

- **The ledger is the behavioural test.** A screenshot cannot show that a double click sent two requests, or that an image beacon fired. The ledger records `execute_attempts` (with `http_status` and `outcome`), `executions` and `security_beacons`. `npm run check` runs the same assertions we run.
- We read the code with the same questions we ask of production code: where do the dangerous decisions live, and how easy is it to get them wrong in the next change?

Submission

- A Git repository and a README covering: architecture (state, streaming, rendering pipeline), how you validate responses and why, your confirmation lifecycle (a state diagram is welcome), your approach to untrusted content, accessibility decisions, and "what I would do with more time / what changes for production".
- The client in `client/` of this repository, with `mock-server/`, `data/`, `schema/`, `scripts/` and `scenarios.jsonl` unmodified. Instructions to run it: ideally one command next to `npm start` for the mock.
- A few screenshots or a short screen recording of rows 4–9.

Constraints and notes

- **TypeScript** is required. The framework is your choice; React is what we use and is preferred. We do not hold another choice against you. What we assess is your approach and your reasoning.
- Libraries are your choice (markdown, validation, state, styling). Choosing a library is a decision like any other: we will ask about the defaults that matter. What does your markdown library do with raw HTML? What does your validator do with extra fields?
- You are free to use AI tools such as Claude Code or Cursor. What we measure is reasoning and architecture, and whether you understand every line you submit.
- Time: one weekend, roughly 3 days. Deliberately narrowing the scope earns credit; completing a smaller, more critical part well beats touching everything and finishing none of it. If you must cut, cut from the bottom of this list, never the top: the stream parser; the renderer and its validation; the confirmation lifecycle; untrusted content; freshness after actions; the audit inspector; then polish and the bonus list. Keyboard operation is not a layer to cut: build it in as you go.
- This README is the specification; the mock is a reference implementation of its server side. If the two disagree, tell us which one you followed and why.
- If you find something ambiguous, make a reasonable assumption and note it in your README.